

Introduction

This is a hands-on lab on knowledge graphs. The objectives of the lab are threefold:

- To learn how to model RDFS knowledge graphs following best practices,
- To understand the differences between property graph and knowledge graph modeling,
- To gain familiarity with a knowledge graph database.

We will use GraphDB¹ for working with knowledge graphs. To prepare for the session, you are expected to have studied the material covered in the lectures (Knowledge Graph foundations, RDF, and RDFS), as well as the SPARQL materials provided for this lab.

Creating Groups

Groups must consist of two students and should be created using the team creation tool in Learn-SQL. Please register your group accordingly. The only constraint is that you may not work with the same partner as in the first lab. Before the deadline, upload your group's lab solutions to Learn-SQL (see the corresponding deliverable event).

What to Deliver?

One group member must submit the solutions (see below for a precise description of the required deliverables) and include the names of all group members in the document. Please check the assignment deadline carefully and ensure that it is met, as it is strict. Upload a single compressed file containing your solutions. The main file must be a PDF named [Group] - [MemberSurname].pdf (e.g., G11A-QueraltRomero.pdf). The document must be structured according to the sections described below, and within each section, you should present your solution following the given instructions. If you make any assumptions not explicitly stated in the problem, include a note in the corresponding section.

The compressed file must also include the two additional files required (the Python and ttl files; more details below).

¹<https://graphdb.ontotext.com/>

A Knowledge Graph Modeling

Graph modeling is essential for several reasons. In addition to capturing the appropriate domain concepts and relationships, well-structured semantics ensure data interoperability and integration across heterogeneous sources, facilitate reuse and extensibility of the model, and improve query expressiveness and maintainability. Moreover, a well-designed model enables automated reasoning and inference, supports data quality validation, and reduces ambiguity, leading to more robust and trustworthy downstream applications, such as Retrieval-Augmented Generation (RAG) with LLMs, semantic search, recommendation systems, knowledge discovery and data integration.

In the first part of the lab, we will promote best modeling practices and highlight the differences between property graphs and knowledge graphs. In particular, we will examine why property graphs are primarily designed to assert facts and tend to fall short when representing general knowledge and enabling inference.

Biomedical Data Modeling

We want to build a small data model to capture basic biomedical knowledge about how drugs interact with diseases.

At a minimum, we need to represent **drugs** and **diseases**, and distinguish between different types of each. For **drugs**, we care about things like **anti-inflammatory drugs** and **antibiotics**. For **diseases**, we want to differentiate, for example, **inflammatory diseases** and **infectious diseases**.

More importantly, we need to capture how drugs relate to diseases. A drug can **treat a disease**, **relieve it**, or in some cases even **worsen it**. These are different kinds of effects, but from a higher-level perspective they are all ways in which a drug affects a disease. We would like this general notion of “**affecting**” to be reflected in the model, not just the specific cases.

We also want to include a few data instances. For instance, **Ibuprofen** should be represented as an anti-inflammatory drug, and **Amoxicillin** as an antibiotic. On the disease side, we want examples like **Arthritis**, **Bacterial Infection**, and **Gastric Ulcer**. The model should capture that **Ibuprofen relieves Arthritis**, **Amoxicillin treats Bacterial Infection**, and that **Ibuprofen can worsen Gastric Ulcer**.

The idea is to keep the dataset small, but structured enough so that we can see both the hierarchy of concepts (types of drugs and diseases) and the relationships between them, including how more specific interactions relate to more general ones.

A team working on Property Graphs, generated the following solution in Neo4J (snippet below):

```
CREATE
  (ibuprofen:Drug:AntiInflammatory {name: "Ibuprofen"}),
  (amoxicillin:Drug:Antibiotic {name: "Amoxicillin"}),

  (arthritis:Disease:InflammatoryDisease {name: "Arthritis"}),
  (bacterialInfection:Disease:InfectiousDisease {name: "
    Bacterial Infection"}),
  (gastricUlcer:Disease {name: "Gastric Ulcer"});
```

```
MATCH
  (ibuprofen:Drug {name: "Ibuprofen"}),
  (amoxicillin:Drug {name: "Amoxicillin"}),
  (arthritis:Disease {name: "Arthritis"}),
  (bacterialInfection:Disease {name: "Bacterial Infection"}),
  (gastricUlcer:Disease {name: "Gastric Ulcer"})
CREATE
  (ibuprofen)-[:RELIEVES]->(arthritis),
  (amoxicillin)-[:TREATS]->(bacterialInfection),
  (ibuprofen)-[:WORSENS]->(gastricUlcer);
```

Tasks

Propose a RDFS Knowledge Graph model following the best practices presented in the lectures. You can use any drawing tool you like (e.g., drawio, arrows.app, gra.fo, yED, etc.) but the Knowledge Graph must be correct (i.e., the triples should follow the syntax and semantics introduced in the lectures).

1. Include in the main document ([Group]-[MemberSurname]+.pdf) a section A including the graphical representation of your solution. Clearly distinguish the metamodel, model and facts layers,
2. Justify the modeling decisions made, and
3. Identify the modeling aspects covered in your solution that were not duly covered by the Neo4J solution.

B Knowledge Graphs in GraphDB

Get familiar with GraphDB by following the setup instructions and tutorials in the Appendix.

Creating the KG and Inserting Data

In this section, you are required to create both the RDFS model (general knowledge) and the instance data (facts) in GraphDB.

First, create a GraphDB repository. Choose a *Repository ID* and a *Repository title* of your preference. Make sure to enable inference by selecting *RDFS (Optimized)* under the *Ruleset* option. This setting instructs GraphDB to perform reasoning based on an optimized version of the RDFS entailment regime.

To better understand how GraphDB handles reasoning, you may consult the following resources:

- <http://graphdb.ontotext.com/documentation/free/inference.html#inference-in-graphdb>
- <http://graphdb.ontotext.com/documentation/standard/reasoning.html>
- <https://graphdb.ontotext.com/documentation/free/rdfs-and-owl-support-optimisations.html>

The first link explains how inference can be customized in GraphDB, the second describes the reasoning process and how GraphDB materializes inferred triples, and the third clarifies the differences between the standard RDFS rules presented in the lectures and GraphDB's *optimized* version.

GraphDB does not provide a convenient interface for directly creating a Knowledge Graph beyond the **use of SPARQL CONSTRUCT queries**. Therefore, you are encouraged to generate the graph programmatically (e.g., using **RDFLib**²) and export it to a file (e.g., in **.ttl** format), which can then be loaded into GraphDB.

RDFLib is simple to use yet highly expressive: **you create a graph, add triples to it, and serialize it into a file**. Since inference is enabled in GraphDB, you can avoid explicitly (i.e., manually) creating many triples. A key quality criterion of your solution is to minimize redundancy by not asserting triples that can be inferred from the schema.

For the sake of a comprehensive exercise, consider **extending the model by adding new subclasses to the Drug and Disease classes, as well as approximately 100 instances, in order to have a sufficiently rich dataset to explore and query the Knowledge Graph in GraphDB**.

²<https://rdflib.readthedocs.io/en/stable/>

Tasks:

1. Deliver your Python code generating the RDFS Knowledge Graph. You must also include the .ttl file(s) you generated and loaded to GraphDB. Name both files as follows: [Group]-[MemberSurname]+.py and [Group]-[MemberSurname]+.ttl.

C Querying the Knowledge Graph in GraphDB

Once the Knowledge Graph has been created and loaded into GraphDB it is ready to be queried. We are specifically interested in the benefits of having an explicit model defined and enabling reasoning.

Tasks

1. Include in the main document a Section B. In the main PDF file, provide the following SPARQL queries (explicitly stating any assumptions you make):
 - List all drugs.
 - List all diseases.
 - Retrieve all pairs of drugs and diseases such that the drug affects the disease.
2. For each query, compare it with its counterpart in the Neo4j graph provided, and justify the differences.
3. For each query, explain the inferences involved and the triple patterns generated by GraphDB. Is this behaviour what you expected? Justify your answer.

APPENDIX. GraphDB Setup and Tutorials

To setup GraphDB follow these steps:

- Download the Graph DB installer: <https://www.ontotext.com/products/graphdb/graphdb-free/>
Fill in the form that appears on the right. After submitting the form, you will receive an email at the address you provided with various download links depending on your OS (Windows, MacOS or Linux). Please also check your junk / spam folder in case the email went there.
- Install GraphDB: After downloading the installer, you should run normally and you do not require an administrator account to run it. Follow the steps that appear in the executor without changing any of the configurations and wait for the installation process to finish.
- Launch GraphDB: Once the installation is complete, start the GraphDB server by launching the **GraphDB Desktop** application from your home or start menu. When the server is up and running, a browser window should open automatically at <http://localhost:7200/>. Make sure to install the license file you received by email.
- Configure settings: Go to the GraphDB Desktop application (i.e., the background stand-alone window, not to the browser) and click on the button ‘**Settings...**’ at the top. A new pop-up window should open, and you should introduce the following custom java properties (the first one sets the maximum number of triples that can be loaded via GraphDB Workbench and the heap space used by the JVM):

```
graphdb.workbench.maxUploadSize=40000000000
```

```
Xmx=2000m
```

Afterwards, click on the ‘**Save and restart**’ button at the bottom right. The GraphDB server will restart and will open a new browser window again with the same URL as above.

Follow the comprehensive tutorials provided by GraphDB to get familiar with the tool:
<http://localhost:7200/guides>